FAQ-03: OneAgent vs OpenTelemetry for Java/Scala — Convert, Layer, or Leave Alone?

> **Series:** FAQ — Frequently Asked Questions | **Reference:** 03 — OneAgent vs OpenTelemetry for Java/Scala | **Created:** May 2026 | **Last Updated:** 05/07/2026

Overview

The recurring question:

Teams already running OpenTelemetry on their Java, Scala, Kotlin, or Groovy backends frequently ask two related questions: *Is it worth the effort to convert our existing OTel-instrumented services to Dynatrace OneAgent?* And the sharper version: *If our OTel implementation is working today, do we need OneAgent at all?*

Both come down to a level-of-effort versus capability-gain trade-off, and both deserve a frank answer.

Short answer (effort-vs-conversion): If an application is already emitting OpenTelemetry data and is not running on serverless, **the most cost-effective path is almost always to keep OTel for application instrumentation and add OneAgent alongside it for host, JVM, and infrastructure coverage** — not to convert. The two are complementary, not competing. A wholesale conversion is usually a *deletion* exercise, not a re-instrumentation exercise — and even then, the gain is rarely worth the disruption unless a specific Dynatrace capability is needed that OTel cannot provide on its own.

Short answer (is OneAgent required at all): No, not strictly. OTel alone can fully instrument an application and ship to Dynatrace via OTLP — Davis AI works on that data, traces correlate via W3C `traceparent`, and metrics/logs land in Grail. **But OTel-only leaves real gaps** in host metrics, JVM internals, Smartscape topology, code-level method tracing, and RUM-to-backend stitching. Whether those gaps matter depends on the workload — see §2 for the side-by-side capability table.

This FAQ frames the trade-offs, the hidden costs, and the workload-specific edge cases (especially **effect systems** like ZIO, Cats Effect, and fs2 where async concurrency materially changes the answer).

Table of Contents

Prerequisites

Requirement	Details
Audience	Engineering leads, SRE/observability architects, platform owners evaluating instrumentation strategy
Format	Decision-support FAQ — no DQL or hands-on labs

Requirement	Details
Assumed knowledge	Familiarity with the basics of distributed tracing, the difference between an *agent* and an *SDK*, and what your services run on (JVM, container, serverless)
Scope	JVM ecosystem (Java, Scala, Kotlin, Groovy, Clojure). Many points generalize to other runtimes, but Scala-specific notes apply only to JVM Scala (not Scala.js or Scala Native)

1. Framing the Question Correctly

Customers usually ask this as **OneAgent OR OpenTelemetry** — a binary choice. That framing is the first mistake. The two tools answer different questions:

- **OneAgent** answers: *"What is happening on my hosts, in my JVM, and in the frameworks I'm running — without me writing any instrumentation code?"*
- **OpenTelemetry** answers: *"How do I emit portable, vendor-neutral, code-level telemetry from my application — including the parts no agent can see automatically?"*

Re-framed correctly, the question becomes: **"Given we already have OTel, do we still need OneAgent? And if we add OneAgent, do we need to remove OTel?"** The answer to the first is *usually yes* (for non-serverless workloads). The answer to the second is *almost never*.

2. Is OneAgent Required at All?

Short answer: No, not strictly — but most non-serverless teams are still better off with both.

OpenTelemetry alone can fully instrument an application and ship to Dynatrace via OTLP. Traces correlate via W3C `traceparent`, metrics and logs land in Grail, and Davis AI operates on that data. For serverless workloads, effect-system Scala (ZIO / Cats Effect / fs2 / http4s on IO), or a vendor-portability mandate, **OTel-only is the right answer** — OneAgent adds nothing critical, or in the case of serverless cannot run at all.

Where OTel-only leaves real gaps (and why most non-serverless teams add OneAgent anyway):

Capability	OTel-only	Requires OneAgent
Application traces / metrics / logs	Yes	—
JVM internals (heap, GC, threads, classloaders)	Partial — needs JMX receiver or runtime-telemetry library, wired per service	Yes — automatic
Host metrics (CPU, memory, disk, network)	Partial — needs OTel Collector with hostmetrics receiver deployed on every node	Yes — automatic
Process / container correlation	Manual attribute hygiene	Yes — automatic
Smartscape topology and dependency map	Not available	Yes
PurePath code-level method tracing on demand	Not available	Yes
Real-User Monitoring ↔ backend trace stitching	Manual wiring	Yes — automatic
Auto-instrumentation of frameworks OTel doesn't cover (proprietary middleware, exotic app servers)	Gap	Yes — often covered
Davis AI fidelity	Good if attributes are clean and consistent	Best — Davis is purpose-built for OneAgent's data model
Update cadence	Per-application redeploy	Centrally managed; no app redeploy

Decision rule in three lines:

- **Serverless, multi-backend mandate, or effect-system Scala** → OTel only. OneAgent doesn't help, or can't run.
- **Standard JVM workloads on long-lived hosts/containers, and Smartscape + Davis at full fidelity matters** → add OneAgent alongside OTel. The install is a deployment task — no code change — and the two coexist by design (see §8 for the Span Sensor vs OTLP patterns).

- **Cost-constrained or instrumentation-fatigued, and OTel is working today** → stay OTel-only. No telemetry is missing — only some platform-level conveniences. Re-evaluate if and when those gaps cause real pain.

Honest framing: OneAgent is not *required*, it is an *accelerator*. It buys infrastructure coverage and topology that would otherwise have to be built manually with the OTel Collector, hostmetrics receivers, JMX wiring, and a clean attribute strategy. If those are already in place and working, the marginal value of adding OneAgent is much smaller — though never zero, because Smartscape, PurePath, and the Davis-AI-on-OneAgent-data path have no direct OTel equivalents.

3. What Each Tool Actually Is

OneAgent is a Dynatrace-installed binary that runs on the host (or as a container sidecar/initcontainer in Kubernetes via the Dynatrace Operator). It auto-instruments JVMs by attaching at process startup *or* dynamically into already-running JVMs via the JDK Attach API (`jdk.attach`) — no restart required to begin monitoring an existing process. Once attached, it produces *PurePath* traces, host metrics, JVM metrics, log streams, process snapshots, and Smartscape topology — all without code changes.

OpenTelemetry (OTel) is a CNCF specification with two main consumption modes on the JVM:

1. **OTel Java Agent** (`opentelemetry-javaagent.jar`) — most commonly launched via the `-javaagent:` JVM argument at startup, but the same agent JAR can also be attached at runtime to an already-running JVM via the JDK Attach API (the OpenTelemetry project ships an attach loader for exactly this case). Auto-instruments a similar (but not identical) list of frameworks. Vendor-neutral; emits OTLP.
2. **OTel SDK / API in code** — explicit imports of `io.opentelemetry.api` to create custom spans, metrics, log records, and baggage from inside your application. This is what Scala libraries like *otel4s* and *zio-telemetry* wrap.

These two OTel modes are independent — you can use either, both, or neither. "Already instrumented with OTel" usually means a mix: the Java agent for HTTP/DB instrumentation plus some manual SDK calls for business logic.

The Dynatrace OneAgent SDK for Java (a separate, narrow library — see Dynatrace/OneAgent-SDK-for-Java) is *not* OneAgent itself. It is a thin API for adding custom traces inside an application that is *already running under OneAgent*. With no

OneAgent attached, the SDK calls become no-ops. Its current version is 1.9.0, and it explicitly recommends OpenTelemetry for serverless workloads where OneAgent cannot run.

Practical implication of runtime attach: for both tools, "instrumenting a service" does not strictly require a JVM restart on a long-lived process — the JDK Attach API lets either agent be loaded into a running PID. In day-to-day operations, OneAgent's host-resident model takes advantage of this routinely (newly started or already-running JVMs on a OneAgent-equipped host are picked up automatically), while OTel deployments typically prefer `-javaagent:` for reproducibility but can use runtime attach when a restart is not feasible.

Concrete example — attaching the OTel Java agent to a running JVM:

```
import com.sun.tools.attach.VirtualMachine;

public class DynamicAttacher {
    public static void main(String[] args) throws Exception {
        String pid = args[0]; // Target JVM PID
        String agentPath = "/path/to/opentelemetry-javaagent.jar";
        VirtualMachine vm = VirtualMachine.attach(pid);
        vm.loadAgent(agentPath, "otel.service.name=my-service");
        vm.detach();
    }
}
```

The same `VirtualMachine.attach(pid)` → `loadAgent(...)` pattern works for any compliant JVM agent. OneAgent uses an equivalent attach mechanism internally (driven by the host-resident OneAgent process rather than an operator-launched loader); the OTel Java agent exposes it as a public path for operators who cannot restart the target process. Caveats: the attaching process needs OS-level permission to access the target PID (typically same-user or root), and a small number of `java.lang.instrument` capabilities (e.g., `appendToBootstrapClassLoaderSearch` on some JDKs) behave differently for runtime-attached agents than for `-javaagent:`-loaded ones — reproducibility is the main reason `-javaagent:` remains the default in CI/CD pipelines.

4. Capability Comparison

Dimension	OneAgent	OpenTelemetry (Java agent + SDK)
Owner	Dynatrace (vendor)	CNCF / OpenTelemetry community (vendor-neutral)
Backend	Locked to Dynatrace	Any OTLP-compatible backend (Dynatrace, Jaeger, Tempo, Datadog, New Relic, Splunk, Honeycomb, Grafana Cloud, etc.)
Runtime model	Native binary on host or in container; attaches to JVMs at startup *or* dynamically into running JVMs via the JDK Attach API	Java agent JAR — typically launched via `-javaagent:` at startup, but can also attach at runtime via the JDK Attach API; and/or in-code SDK
Signals captured	Traces, JVM metrics, host metrics, process metrics, log streams, topology, real-user, deep code-level (PurePath)	Traces, metrics, logs, baggage, context (signals you choose to emit)
Auto-instrumented frameworks (JVM)	Hundreds of frameworks/libs/app servers; Akka HTTP 10.1–10.7, Akka Remoting 2.3–2.7, Play Framework 2.2–2.8, plus all major JDBC drivers, message brokers, web servers	A different (overlapping) list: Akka Actors 2.3+, Akka HTTP 10.0+, Play MVC 2.4+, Play WS, Spark 2.3+, Finatra 2.9+, Scala ForkJoinPool 2.8+, plus extensive Java framework coverage
Host / JVM / process metrics	Yes — included automatically	Only if you add the host-metrics receiver or run an OTel Collector node-exporter; not part of basic SDK
Smartscape / topology / dependency map	Yes — automatic	Not provided by OTel; can be approximated from spans but no real-time topology graph

Dimension	OneAgent	OpenTelemetry (Java agent + SDK)
Davis AI / problem detection / RCA	Yes — built into Dynatrace platform on OneAgent telemetry	Available when OTel data is ingested into Dynatrace; quality depends on attribute completeness
Serverless (AWS Lambda, Azure Functions)	Not supported (per OneAgent SDK README)	Fully supported
Update model	Centrally managed by Dynatrace; no app redeploy	Application redeploy required for SDK; agent JAR update requires JVM restart
Java version floor	OneAgent SDK: JRE 1.6+. OneAgent itself: JRE 8+ on most modern releases	OTel Java SDK: Java 8+ (Android needs library desugaring)
Code change required	None for auto-coverage	None for Java agent; explicit imports for custom spans/metrics
Effect-system / fiber awareness	Thread-local context — fragments under fiber-based concurrency	Effect-system specific libraries (otel4s, zio-telemetry) handle fiber context correctly
Release cadence	Monthly (Dynatrace SaaS sprints)	Monthly minor releases (OTel Java 1.61.0 as of writing)

5. What "Convert to OneAgent" Really Means

When customers say *"convert from OTel to OneAgent"*, they often imagine a 1:1 re-instrumentation effort comparable to the original OTel rollout. **It is not.**

For workloads where OneAgent's automatic coverage is sufficient (the common case — REST/gRPC services on Akka HTTP, Play, Spring, plain `java.util.concurrent` thread pools), "converting" looks like this:

1. **Install OneAgent** on the host or via the Dynatrace Operator on Kubernetes. Restart the JVMs. Done — instrumentation is live.

2. **Optionally remove the OTel Java agent** (`-javaagent:opentelemetry-javaagent.jar`) and any OTel SDK setup code. This step is *deletion*, not authoring.
3. **Decide what to do with custom OTel spans/metrics/logs** in code:
   - **Easiest path:** leave them in place — Dynatrace ingests OTel data via OTLP, and OneAgent's *OpenTelemetry Span Sensor* stitches them into the same trace as OneAgent's auto-instrumented spans. (Caveat: do not enable both Span Sensor and OTLP export simultaneously, or you will get duplicate spans.)
   - **Replacement path:** rewrite each `Tracer` / `Meter` / `Logger` call against the Dynatrace OneAgent SDK for Java. This is the only step that involves real code work — and it is **only worth doing for code paths the OneAgent SDK can express that OTel cannot, which is essentially nothing.** The OneAgent SDK is intentionally narrow (incoming/outgoing remote calls, custom services, web requests, messaging, SQL, custom request attributes) and does not cover metrics or logs at all.

For workloads where OneAgent's automatic coverage is insufficient (effect-system Scala, exotic frameworks, message brokers OneAgent doesn't auto-instrument), removing OTel is actively harmful — you would be deleting working instrumentation and replacing it with nothing.

The realization: "Convert to OneAgent" is rarely a re-instrumentation project. It is an *agent installation* project, possibly followed by *deleting OTel agent configuration* if you want to simplify the runtime stack. The custom OTel code in the application generally stays exactly where it is.

6. Framework Auto-Instrumentation Coverage (Java + Scala)

Coverage is the single biggest factor in whether *adding* OneAgent gets you what you want or leaves you patching gaps with the SDK.

Both auto-instrument (overlap zone — most teams gain immediate value from OneAgent here):

- **Akka HTTP** (server + client)
- **Play Framework** (MVC + WS)
- **Servlet / JAX-RS / Spring MVC / Spring Boot** (standard Java web layer)
- **JDBC drivers** (Postgres, MySQL, Oracle, SQL Server, MS-SQL, MariaDB, etc.)

- **gRPC, OkHttp, Apache HttpClient, Java 11+ HttpClient**
- **Kafka, RabbitMQ / AMQP, JMS, ActiveMQ**
- **Standard Java executors /** `ForkJoinPool` **/** `CompletableFuture` (thread-local context)

OTel covers, OneAgent does not (or only partially):

- **Apache Spark Web Framework** — listed in OTel supported-libraries; not in OneAgent's Scala framework support matrix
- **Finatra** (Twitter HTTP framework) — OTel 2.9+; not in OneAgent matrix
- **otel4s / zio-telemetry / fiber-aware tracing** — OTel via Scala-specific libraries (see §6)

OneAgent covers, OTel does not (or only partially):

- **Smartscape topology and process-group correlation** — these are Dynatrace constructs, not standard signals
- **Real-User Monitoring (RUM) ↔ backend trace stitching** — requires OneAgent on the backend tier
- **Code-level method tracing on demand** (PurePath capture without code changes) — not part of the OTel spec
- **Host, process, and JVM-internal metrics** without a separate Collector deployment

Neither covers automatically (custom code or community plugin required):

- **http4s** — neither OneAgent nor the OTel Java agent supported-libraries list includes http4s. Coverage requires either the http4s middleware in *otel4s*, or manual instrumentation against the OneAgent SDK (and the OneAgent SDK has no native fiber-context model — see §6).
- **Tapir, Endpoints4s, sttp** (Scala HTTP DSLs) — typically rely on an underlying http4s/Akka HTTP implementation; coverage follows whichever backend you choose.
- **Many Scala-native streaming and effect libraries** — fs2, Monix, Cats Effect IO.

7. Effect Systems: ZIO, Cats Effect, fs2, http4s

This is the single most important section for Scala teams. It is also where the "OneAgent or OTel" choice has a clear-cut technical answer — not a preference.

The mechanic: classical JVM tracing (both OneAgent's auto-instrumentation and the OTel Java agent's default behavior) propagates trace context using `ThreadLocal`. As long as a request stays on a single thread, or hops across thread pools that the agent has hooked, context flows correctly.

Effect systems break this assumption. ZIO and Cats Effect schedule work on lightweight *fibers* that are multiplexed onto a small thread pool. A single logical operation may run on dozens of different OS threads over its lifetime, and the runtime explicitly does not preserve `ThreadLocal` across fiber suspension/resumption — that would defeat the purpose of fibers. fs2 streams and http4s routes (when run on a Cats Effect IO or ZIO runtime) inherit the same property.

Symptoms when classical tracing meets effect systems:

- A trace begins on a request, then "loses" context after the first `IO.flatMap` or `ZIO.flatMap` that yields
- Spans appear as orphans (no parent) downstream of an async boundary
- Database calls, HTTP calls, and message publishes inside an effect do not link back to the parent request span
- The trace waterfall looks plausibly populated but is structurally wrong

The OTel ecosystem solution:

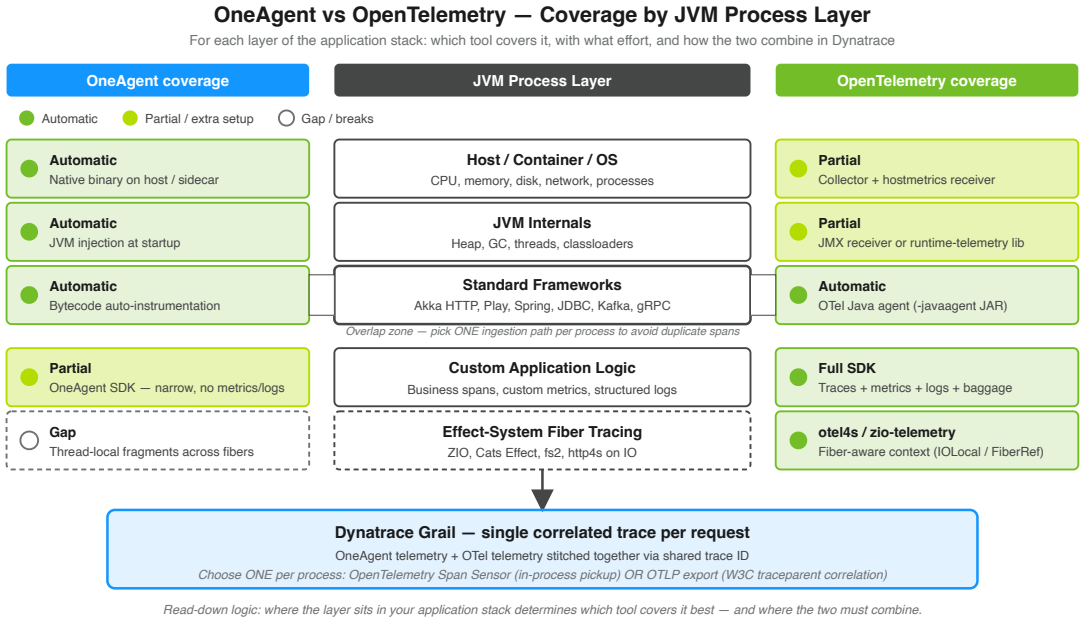
- **otel4s** (typelevel/otel4s) — Typelevel's OpenTelemetry implementation that *"aims to fully and faithfully implement the OpenTelemetry Specification atop Cats Effect."* Context is carried in `IOLocal`, which Cats Effect *does* preserve across fiber boundaries.
- **zio-telemetry** (zio/zio-telemetry) — provides the `zio-opentelemetry` module. Context is carried in `FiberRef` (ZIO's fiber-scoped reference type), which propagates correctly across all ZIO operations.

OneAgent's position on effect systems: OneAgent's auto-instrumentation has no equivalent of `IOLocal` or `FiberRef`. For pure-effect codebases, OneAgent will produce telemetry — JVM metrics, host metrics, OS-level network calls — but the application-level trace structure inside an effect graph will be fragmented or missing. The Dynatrace OneAgent SDK can add manual spans, but it has the same `ThreadLocal`-based context model and will fragment in the same way unless you wrap every effect boundary by hand.

Practical guidance:

Scala stack	Recommended app-level tracing	Recommended infra-level coverage
Akka, Play, Spark, plain `Future` / `ForkJoinPool`	OneAgent auto-instrumentation (no code change)	OneAgent (same)
ZIO + zio-http / sttp on ZIO	**zio-telemetry (OTel)**	OneAgent for host/JVM
Cats Effect + http4s + fs2	**otel4s (OTel)**	OneAgent for host/JVM
Mixed (some Akka services, some ZIO)	Both — pick per service	OneAgent everywhere

8. The Coexistence Pattern (How They Work Together in Dynatrace)



Dynatrace is explicit that the two are designed to coexist. From the Dynatrace OneAgent + OpenTelemetry documentation: *"You can implement this service by service, adopting the open standards of OpenTelemetry where openness matters most, while leveraging enhanced OneAgent features available where you need them."*

The mechanism — OpenTelemetry Span Sensor: when this code-module setting is enabled, OneAgent detects in-process OpenTelemetry API calls and weaves them into

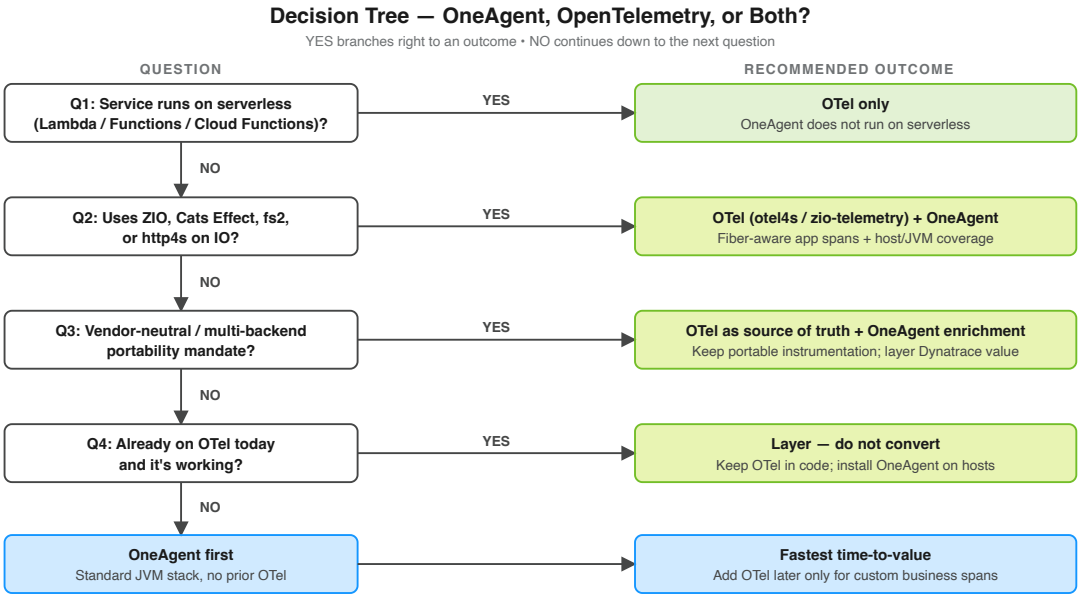
the same trace as OneAgent's auto-instrumented spans. Per the Dynatrace docs: *"These auto-instrumented spans are woven together with your manual OpenTelemetry spans into a single trace."* The result is a unified PurePath-style trace that contains both OneAgent and OTel-emitted spans, correlated through a shared trace ID.

Critical caveat: if you enable the OpenTelemetry Span Sensor *and* simultaneously export OTLP from the same process, **you will get duplicate spans** — once from the Span Sensor's in-process pickup, once from the OTLP exporter. Pick one ingestion path per process. The two supported patterns are:

1. **Span Sensor only** — OTel API calls in code, no OTLP exporter configured. OneAgent picks them up locally, ships them inside its native channel.
2. **OTLP only** — Span Sensor disabled, OTel exporter configured to send to a Dynatrace OTLP endpoint (or to an OTel Collector forwarding to Dynatrace). OneAgent traces and OTel traces correlate via standard W3C `traceparent` propagation.

Either pattern produces a single, correlated trace in Dynatrace. Customers regularly run both OneAgent and OTel in the same process precisely because the platform was built to merge them.

9. Decision Tree by Workload Type



Plain-language flow:

1. **Serverless?** → OTel only. OneAgent does not run on AWS Lambda, Azure Functions, GCP Cloud Functions, or similar. The OneAgent SDK README explicitly directs serverless users to OpenTelemetry.
2. **Effect-system Scala (ZIO / Cats Effect / fs2 / http4s on IO)?** → OTel via *otel4s* or *zio-telemetry* for application spans. Add OneAgent for JVM/host signals if the workload runs on long-lived hosts or containers.
3. **Multi-backend mandate, regulatory portability, or active exit-ramp planning?** → OTel as your primary instrumentation. OneAgent becomes value-add, not the source of truth.
4. **Standard JVM stack (Akka, Play, Spring, plain `Future`)?** → OneAgent's auto-instrumentation gives you the fastest time-to-value. If you already have OTel, keep it for custom spans; if not, you may not need it at all.
5. **Already instrumented with OTel and it's working?** → Add OneAgent alongside. Do *not* rip out OTel. The combined cost is lower than the conversion cost, and you keep your portability.

10. Three Migration Paths and Their Effort

Path A: Stay OTel-only (no OneAgent at all)

Aspect	Reality
Effort	Zero — keep current state
Code changes	None
What you keep	Vendor neutrality, fiber-aware tracing on effect systems, serverless coverage, single instrumentation model across polyglot stacks
What you give up	Smartscape topology, automatic JVM/host/process metrics without a Collector, on-demand code-level method tracing, deepest Davis AI fidelity, RUM-to-backend stitching unless you wire it manually
Best for	Greenfield CNCF-aligned organizations; serverless-heavy estates; multi-cloud/multi-vendor strategies; effect-system-dominant Scala teams

Path B: Add OneAgent alongside OTel (the layered pattern — recommended for most)

Aspect	Reality
Effort	Low — install OneAgent on hosts or via Dynatrace Operator, restart JVMs
Code changes	None — keep all existing OTel code
What you gain	Host/JVM metrics, Smartscape, Davis AI, automatic framework coverage where it overlaps with OTel, RUM stitching
What you must decide	Span Sensor vs OTLP export (pick one per process — see §7 caveat)
Risk	Duplicate spans if both ingestion paths active simultaneously; resolved by configuration
Best for	Teams already on OTel who want Dynatrace's infra and AI features without losing their portable instrumentation

Path C: Rip-and-replace OTel with OneAgent (and OneAgent SDK for custom spans)

Aspect	Reality
Effort	Highest — install OneAgent, remove OTel agent + dependencies, rewrite every custom span/metric/log against the OneAgent SDK
Code changes	Per-service rewrite of every `Tracer.spanBuilder(...)`, every `Meter.counterBuilder(...)`, every OTel `Logger.emit(...)` to OneAgent SDK equivalents — and the SDK does not cover metrics or logs at all, so those signals are lost or must be re-emitted via Dynatrace metrics ingest APIs
What you gain	One instrumentation model in code; tighter native PurePath semantics in places
What you lose	Vendor neutrality, broad community ecosystem, fiber-aware libraries on effect systems, serverless coverage, parts of your metrics/logs surface
Risk	High — coordinated multi-service rewrite, regression-prone, must be repeated for every new service
Best for	Almost no one. Defensible only if (a) the engineering org has a strict single-vendor mandate, (b) all workloads are non-serverless and non-effect-system, and (c) the cost of dual maintenance demonstrably exceeds the rewrite cost

11. Pros and Cons Side-by-Side

OneAgent — pros

- Zero-code instrumentation for hundreds of frameworks
- Host, JVM, process, network, and topology coverage out of the box
- Smartscape dependency map and Davis AI tuned to its data model
- RUM-to-backend trace correlation
- Centrally managed updates — no application redeploy
- Operations-team-friendly (instrument an environment without engineering involvement)

OneAgent — cons

- Locked to Dynatrace as a backend
- Does not run on serverless functions
- Thread-local context model fragments under fiber-based concurrency (ZIO, Cats Effect, fs2)
- Custom-span surface (OneAgent SDK) is intentionally narrow — no metrics or log signals
- Framework support is broad but not exhaustive (e.g., http4s, Finatra, Spark not covered automatically)

OpenTelemetry — pros

- Vendor-neutral; works against Dynatrace, Jaeger, Tempo, New Relic, Datadog, Splunk, Honeycomb, etc.
- Full signal coverage — traces, metrics, logs, baggage
- Effect-system-aware libraries (otel4s, zio-telemetry) handle fiber context correctly
- First-class on serverless
- Strong community ecosystem, monthly releases, broad plugin coverage
- Code-level control of sampling, attributes, and span structure

OpenTelemetry — cons

- No automatic host / JVM / topology coverage without a Collector or supplementary agent
- No equivalent to Smartscape or PurePath without backend enrichment
- Requires application redeploy for SDK changes

- Quality of Davis AI and other Dynatrace platform features depends on attribute completeness
- More moving parts (agent JAR, SDK code, Collector) than a single OneAgent install

12. Is the Conversion Worth the Effort?

Question every customer should ask before starting: *"What capability am I gaining that I do not already have, and is it worth the engineering disruption to get it?"*

Run through this checklist honestly:

1. **Does my OTel data already land in Dynatrace via OTLP?** If yes, you already have most of what "converting" would give you for traces.
2. **Am I missing host, JVM, or topology signals?** If yes, *adding* OneAgent solves that — *removing* OTel does not.
3. **Am I using ZIO, Cats Effect, fs2, or http4s on IO?** If yes, removing OTel will *break* tracing — there is no equivalent fiber-aware instrumentation in OneAgent or its SDK.
4. **Do I have any serverless workloads?** If yes, OneAgent cannot run there; you will need OTel anyway, and a hybrid is unavoidable.
5. **Is there an organizational mandate for vendor portability or multi-backend support?** If yes, OTel is a requirement, not a choice.
6. **What is the cost of a coordinated multi-service rewrite of custom spans, metrics, and logs?** Compare honestly against the cost of running both for the foreseeable future.
7. **Is anyone on the team going to maintain a parallel internal instrumentation library against the OneAgent SDK?** If not, even partial Path C is a slow walk into instrumentation rot.

Common false economies:

- *"Removing OTel will reduce overhead."* The OTel Java agent's overhead is already small (typically 1–3% CPU); removing it after installing OneAgent does not measurably move the needle. OneAgent has its own overhead in the same range.
- *"One instrumentation model is simpler."* True at the code level, false at the platform level — Dynatrace was designed to merge both, and customers running both report fewer surprises than customers running one tool stretched beyond its design center.

- *"OneAgent traces are better than OTel traces."* For overlapping framework coverage on standard JVM stacks, the data is comparable. OneAgent's advantage is in the *surrounding* signals (host, JVM, topology, Davis), not in the raw spans.

The honest answer for almost every Java/Scala team already on OTel: the level of effort to *add* OneAgent is small (a deployment-engineering task, not a re-instrumentation task). The level of effort to *replace* OTel with OneAgent is large and almost never repaid by the marginal capability gained. **Layer, don't convert.**

13. Recommended Approach

Default recommendation for a Scala/Java backend already instrumented with OpenTelemetry:

1. **Keep your OpenTelemetry instrumentation in place.** Do not start a conversion project.
2. **Install OneAgent** on the hosts (or via the Dynatrace Operator on Kubernetes) where the services run. Restart the JVMs.
3. **Pick one of the two supported coexistence patterns** to avoid duplicate spans:
   - *Pattern 1 — Span Sensor:* enable the OpenTelemetry Span Sensor in the OneAgent code module; **disable** any OTLP exporter in the application. OneAgent picks up the OTel API calls in-process.
   - *Pattern 2 — OTLP:* keep the OTel exporter pointed at Dynatrace's OTLP endpoint (or an OTel Collector forwarding to Dynatrace); **disable** the Span Sensor. Spans correlate via W3C `traceparent`.
4. **For ZIO / Cats Effect / fs2 / http4s services**, ensure the OTel layer is *otel4s* or *zio-telemetry* (not raw OTel Java SDK), so fiber context flows correctly. Do not attempt to replace this layer with the OneAgent SDK — it does not have an equivalent.
5. **For serverless workloads**, OTel only. OneAgent does not run there.
6. **Do not rewrite custom OTel spans, metrics, or logs against the OneAgent SDK** unless you have a specific Dynatrace capability that the SDK uniquely enables (in practice, this is rarely the case).
7. **Re-evaluate annually** — if Dynatrace adds new effect-system-aware tracing or richer SDK coverage, the calculus may shift. Today, layering is the right answer.

14. Related Technology Map

JVM languages — applicability of this guidance:

Language	Applies as written?	Notes
Java	Yes	Reference case
Kotlin	Yes	Coroutines have a similar fiber-like context issue; OTel's `kotlinx.coroutines` instrumentation handles it. OneAgent's coroutine support is improving but historically has been a gap.
Scala (classical — Akka, Play, plain `Future`)	Yes	OneAgent auto-instrumentation is strong here
Scala (effect systems — ZIO, Cats Effect, fs2)	Modified	See §6 — OTel via *otel4s* / *zio-telemetry* is technically required, not preferred
Groovy	Yes	Tracks Java; auto-instrumented by both
Clojure	Mostly	Same JVM auto-instrumentation; `core.async` channels have context issues comparable to coroutines

Adjacent OTel ecosystem libraries to know about:

- **opentelemetry-java-instrumentation** — the OTel Java agent and broad library auto-instrumentation
- **otel4s** — Typelevel's Cats Effect-native OpenTelemetry implementation
- **zio-telemetry** — ZIO's OpenTelemetry/OpenTracing/OpenCensus integration; `zio-opentelemetry` module
- **opentelemetry-collector** — vendor-neutral telemetry pipeline; common deployment pattern is app → Collector → Dynatrace OTLP

Adjacent Dynatrace capabilities that depend on OneAgent:

- **Smartscape** — automatic topology and dependency mapping
- **PurePath** — code-level distributed trace with method-level granularity on demand
- **Davis AI** — anomaly detection, problem correlation, RCA

- **Real-User Monitoring ↔ backend** — RUM session-to-trace stitching
- **Process/host/log correlation** — automatic by virtue of agent placement

Adjacent topic series in this notebook collection:

- **OTEL series** (`topics/otel/`) — OpenTelemetry integration deep dives
- **AIOPS series** (`topics/aiops/`) — Davis AI, predictive/causal/generative intelligence
- **K8S series** (`topics/k8s/`) — Dynatrace Operator, DynaKube, Kubernetes observability
- **DASH series** (`topics/dash/`) — dashboards over mixed OneAgent + OTel data
- **ADOPT series** (`topics/adopt/`) — instrumentation maturity and adoption roadmap

15. Sources and References

Dynatrace official documentation:

- Dynatrace OneAgent SDK for Java (GitHub) — version 1.9.0; explicit serverless-not-supported note directing to OpenTelemetry; current support matrix
- Use OneAgent with OpenTelemetry data (Dynatrace docs) — OpenTelemetry Span Sensor mechanism, *"auto-instrumented spans are woven together with your manual OpenTelemetry spans into a single trace"*, duplicate-spans warning when both Span Sensor and OTLP export are enabled
- Extend Dynatrace with OpenTelemetry (Dynatrace docs) — *"adopting the open standards of OpenTelemetry where openness matters most, while leveraging enhanced OneAgent features available where you need them"*
- Ingest OpenTelemetry data — getting started (Dynatrace docs) — OTLP endpoints, Collector, Istio/Envoy paths
- Java OpenTelemetry walkthrough (Dynatrace docs) — adding observability to Java applications using OpenTelemetry libraries and tools
- Dynatrace supported technologies — Java/Scala frameworks (Dynatrace docs) — Akka HTTP 10.1–10.7, Akka Remoting 2.3–2.7, Play Framework 2.2–2.8

OpenTelemetry official sources:

- OpenTelemetry Java SDK (GitHub) — current stable 1.61.0; monthly minor releases; Java 8+
- OpenTelemetry Java Auto-Instrumentation (GitHub) — Java agent JAR

- OpenTelemetry Java supported libraries (GitHub) — Akka Actors 2.3+, Akka HTTP 10.0+, Play MVC 2.4+, Play WS 1.0+, Spark Web 2.3+, Finatra 2.9+, Scala ForkJoinPool 2.8+
- OpenTelemetry Java Documentation (opentelemetry.io) — overview, ecosystem, end-to-end examples

JVM platform references:

- JDK Attach API — `jdk.attach` module (Oracle Java 21 docs) — the standard JVM mechanism that allows agents (including OneAgent and the OpenTelemetry Java agent) to attach to an *already-running* JVM, not only at process startup
- `com.sun.tools.attach.VirtualMachine` (Oracle Java 21 docs) — the entry point used by attach loaders

Effect-system OpenTelemetry libraries:

- otel4s (GitHub) — *"aims to fully and faithfully implement the OpenTelemetry Specification atop Cats Effect"*; Typelevel project
- otel4s project site — *"Telemetry meets higher-kinded types"*
- zio-telemetry (GitHub) — `zio-opentelemetry` module; type-safe clients for OpenTelemetry, OpenTracing, OpenCensus
- zio-telemetry documentation — purely-functional context propagation between services

Adjacent Dynatrace community resources:

- Dynatrace Community Examples (GitHub) — community-maintained dashboards, apps, and integrations
- Dynatrace OpenTelemetry on AWS Lambda (Dynatrace docs) — referenced from the OneAgent SDK README as the recommended serverless path

---